

Association Rules Mining: IMDB Movie Data

Tram Anh Hoang^{1*†}

¹Data Science for Economic, Università degli Studi di Milano

15/03/2026

Abstract

This project investigates how different frequent-itemset algorithms perform when mining actor collaboration patterns and success-related associations from the IMDB Top-1000 movies. Using a cleaned version of the dataset, I model each movie as a basket of actors and, in an enriched setting, as a basket of actors plus success metrics and content features (discretised revenue, rating, votes, runtime, and critic-score tiers). I implement PCY (Park-Chen-Yu) and the PCY-Multistage (PMS) algorithm, showing that on actor-only baskets PCY and PMS sharply reduce candidate pairs with modest runtime gains, while on enriched baskets PMS offers both strong pruning and the best overall runtime (up to 90% at higher support thresholds). Using a multi-stage rule mining pipeline based on support, confidence, interest (leverage), lift, Kulczynski index, and Fisher’s exact test, I extract compact sets of statistically significant actor collaboration rules and actor/feature $\rightarrow$ success-tier associations. These findings illustrate how hashing-based improvements and enriched basket design can make association-rule mining both scalable and interpretable on real-world movie data.

1 Introduction

Data mining is the process of discovering useful patterns, correlations, and structure in large datasets, often in support of decision making or prediction tasks. Typical data-mining tasks include classification (predicting labels), clustering (grouping similar objects), anomaly detection (finding unusual cases), and pattern discovery in transactional data such as market baskets. Within this toolbox, association-rule mining, first formalised by Agrawal et al., 1993 for shopping-basket data, is now a standard data-mining task for discovering co-occurring items and “customers who bought X , Y also tend to buy Z ”, making it one of the standard tools for analysing transactional data in retail, web usage, and many other domains (“Mining of Massive Datasets”, n.d.).

The main objective of this project is to understand, in practice, how classical frequent-itemset algorithms Apriori and its hashing-based variants PCY (Park-Chen-Yu) and Multistage PCY actually behave on a real dataset rather than on supermarket basket examples. Instead of treating association-rule mining as a black-box function call, I use the IMDB dataset as a controlled testbed to observe how design choices (support thresholds, basket encoding, and hashing parameters) affect candidate generation, pruning, memory use, and runtime for these algorithms.

To support this goal, I model each movie as a basket of up to four main actors and optionally enrich baskets with discretised success indicators (gross revenue, rating, votes, runtime, meta score) so that Apriori and PCY are stressed both in a simple “actors only” setting and in a slightly more complex, feature-enriched one. Across these scenarios I track quantities like the number of frequent singletons, candidate pairs, truly frequent pairs, and wall-clock runtime, which makes the inner workings and trade-offs of Apriori and PCY visible and comparable on concrete movie data.

Concretely, this project is primarily an experimental study of how Apriori, PCY, and Multistage PCY behave on a real, moderately sized transaction dataset derived from IMDB movies. Rather than proposing a new algorithm, the focus is on making the mechanics and trade-offs of these classic methods visible through systematic measurements on actor-only and feature-enriched baskets.

2 Association Rule Problem

In Association Rule Mining (ARM) for market-basket analysis, the goal is to discover sets of items that occur together frequently and to express them as rules of the form $I \rightarrow j$, where baskets that contain all items in I are likely also to contain item j . This setting is usually formalized by the market-basket model, in which each transaction (T_i) is a small subset of items from a global universe (I). A set of items is called a *frequent itemset* if it appears together in at least a given number of baskets, known as the *support threshold*. This model was originally introduced to analyze supermarket transactions (e.g., bread, milk, diapers, beer), but it has since been applied to domains such as text mining (words as items, documents as baskets) and biomedical data (biomarkers and diseases).

However, this problem formulation immediately raises two practical challenges in large real-world datasets. First, the number of possible *itemsets* ($X \subseteq I$) and rules ($X \Rightarrow Y$) grows exponentially $2^{|I|}$ with the number of items, so a naive approach that evaluates every possible rule is infeasible. This is emphasized in both algorithmic primers and survey papers on frequent itemset mining (Haripreethi & Ananthanath, 2025; “A Primer to Frequent Itemset Mining for Bioinformatics”, n.d.), which note that the search space is exponential and that candidate generation is a central bottleneck. Second, when the transaction database does not fit in main memory (the usual case), algorithms must scan all transactions repeatedly to count supports of different candidate sets. Even with pruning strategies, constrained variants of frequent itemset mining are NP-hard.

To address these limitations, classical level-wise methods such as Apriori from Agrawal et al., 1993 address the first issue by exploiting the monotonicity of frequent itemsets to prune candidates whose subsets are infrequent, but they still generate and test large candidate sets and require one full database scan per itemset size. Hash-based extensions (e.g., PCY and related variants) further reduce the number of candidate pairs that need to be counted by using hash buckets to filter out pairs that cannot be frequent, while alternative approaches such as FP-Growth avoid explicit candidate generation altogether by compressing the transaction database into a prefix tree.

In this project, I apply three representative ARM algorithms: Apriori, PCY, and Multistage PCY to the IMDB Top-1000 dataset, treating each movie as a basket and the four main actors as items. I build two basket representations:

- Actor-only baskets: Each movie contains 4 actors, yielding 1,000 baskets over 2,709 distinct actors
- Enriched baskets: Actors plus discretized tiers for revenue, votes, rating, genre, and decade

My contributions include:

- End-to-end reproducible pipeline from raw CSV to cleaned baskets, frequent itemsets, and association rules
- Empirical comparison of Apriori, PCY, and PCY-Multistage showing expected candidate pair reduction behavior
- Rule mining framework using “interest for filtering, lift for ranking” with Fisher’s exact test for statistical validation

3 Methods

3.1 Dataset Overview

3.1.1 Data Diagnostics

The IMDB Top-1000 dataset contains 1,000 movies with 16 columns including title, release year, runtime, genre, certificate, IMDB rating, Metacritic score, four main actors (Star1–Star4), votes, and gross revenue (Table 1). I standardize column names, parse numeric fields, and derive decade and primary genre features. Missing values are retained rather than imputed, as their absence is structurally linked to film era and origin.

Table 1: Overview of IMDB Top 1000 Dataset Columns

#	Column	Non-null count	Dtype	Description
0	Poster_Link	1000	str	URL to the movie poster
1	Series_Title	1000	str	Movie title
2	Released_Year	1000	str	Release year
3	Certificate	899	str	Age/content rating
4	Runtime	1000	str	Runtime string (e.g., “142 min”)
5	Genre	1000	str	Comma-separated genres
6	IMDB_Rating	1000	float64	IMDb user rating
7	Overview	1000	str	Short plot summary
8	Meta_score	843	float64	Metacritic score
9	Director	1000	str	Director name
10	Star1	1000	str	Main cast member 1
11	Star2	1000	str	Main cast member 2
12	Star3	1000	str	Main cast member 3
13	Star4	1000	str	Main cast member 4
14	No.of.Votes	1000	int64	Number of IMDb votes
15	Gross	831	str	Box office gross (string)

Table 2: Summary Statistics of Movie Dataset Numerical Variables

	Runtime	IMDb Rating	Metascore	No. of Votes	Gross
	(min)				(million \$)
count	1000.00	1000.00	843.00	1.00×10^3	831.00
mean	122.89	7.95	77.97	2.74×10^5	68.03
std	28.09	0.28	12.38	3.27×10^5	109.75
min	45.00	7.60	28.00	2.51×10^4	0.00
25%	103.00	7.70	70.00	5.55×10^4	3.25
50%	119.00	7.90	79.00	1.39×10^5	23.53
75%	137.00	8.10	87.00	3.74×10^5	80.75
max	321.00	9.30	100.00	2.34×10^6	936.66

A multi-panel figure (see Figure 1 and 3) shows the marginal distributions of imdb_rating, runtime, gross, and the frequency of primary_genre, certificate, and decade. This highlights skewed numeric variables and the dominance of a few genres and modern decades and how much support each categorical slice has.

The main data problems visible at this stage are:

Figure 1: Numeric Distribution EDA snapshot

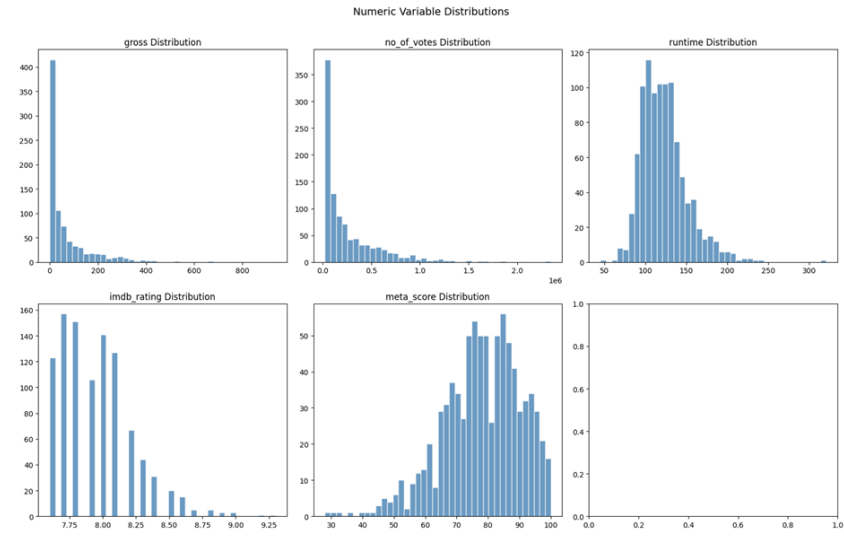


Figure 2: Box plot distribution in numeric variables

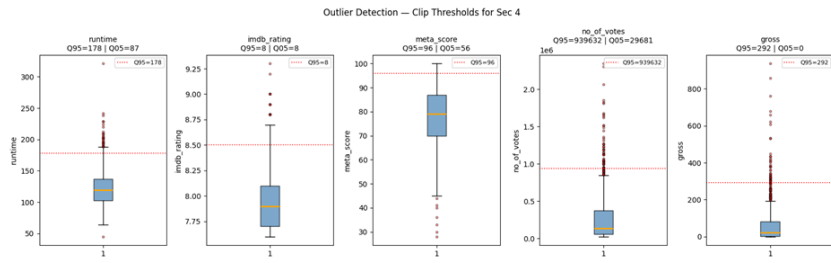
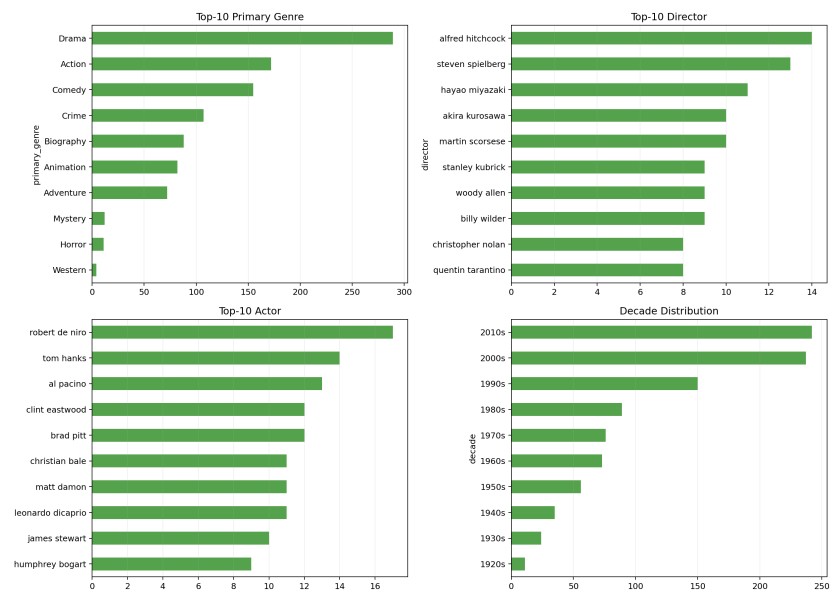


Figure 3: Categorical coverage



- Inconsistent formatting for year, runtime, and gross
- Non-trivial missingness for revenue and external critic scores
- Composite genre strings that are not directly usable as categorical features.

3.1.2 Data Clean

To make numeric fields analysis-ready, I apply the following transformations:

- **released_year**: parse to numeric, coercing errors to NaN; derive **decade** as $\lfloor \text{year}/10 \rfloor \times 10$, then label values as strings such as “1990s”. Non-numeric years (1 row) are retained as NaN and excluded only from temporal summaries.
- **runtime**: extract the numeric part from strings like “142 min” and cast to float minutes.
- **gross**: remove thousands separators, cast to float, and divide by 10^6 to obtain millions of USD.

Rows with missing gross are not dropped; instead, I treat the missingness itself as informative (older and non-US productions) and exclude these rows only from revenue-based analyses. IMDB rating, Meta_score, and number of votes are all coerced to numeric dtypes so they can be used in quantile-based tiering later. (Table 3 3)

Table 3: Skewness and Kurtosis of Key Numeric Variables

Variable	Mean	Median	Skewness	Kurtosis	Missing (%)
runtime	122.89	119.00	1.21	3.43	0.0
imdb_rating	7.95	7.90	1.02	1.43	0.0
meta_score	77.97	79.00	-0.61	0.42	15.7
no_of_votes	273692.91	138548.50	2.30	6.90	0.0
gross	68.04	23.53	3.13	13.91	16.9

For categorical variables, I standardize all column names to lowercase snake_case and clean simple categorical fields. The genre column, which contains comma-separated lists (e.g., Action, Crime, Drama), is split into:

- **genre_list**: a list of cleaned genre labels for each movie,
- **primary_genre**: the first genre in the list, used in most of the EDA and rule interpretation.

After cleaning, the dataset retains all 1,000 films with (as noted in Table 4):

- **Actors**: 2,709 distinct actors, no duplicate rows, normalized to lowercase with trimmed whitespace
- **Average basket size**: 3.996 actors per movie
- **Missing data**: 16.9% for Gross revenue, 15.7% for Meta scores, 10.1% for certificates

Table 4: Categorical Variable Summary

Variable	Observed rows	Unique values	Top category	Top count	Top share (%)
primary_genre	1000	14	Drama	289	28.90
certificate	899	16	U	234	26.03
decade	999	11	2010s	242	24.22

3.2 Association Rule Mining Approaches

As mentioned, in this project, frequent pairs are the core computational bottleneck and the base for downstream rule generation. Candidate-pair growth directly drives runtime and memory use, especially at low support thresholds. The next section will explain in detail how each algorithm is applied in my IMDB experiments.

Considering each basket is one movie and the items are either its four main actors (actor-only) or actors plus discretized feature tiers (enriched). In both settings the item universe is I , with items such as `actor:tom hanks` or `gross_tier:high`, and each movie basket is $T_i \subseteq I$.

Let $T = \{T_1, \dots, T_m\}$ be a multiset of transactions over a universe of items I . A frequent itemset $X \subseteq I$ satisfies:

$$\text{supp}(X) = \frac{|\{i : X \subseteq T_i\}|}{m} \geq \text{minsup}.$$

An association rule $X \Rightarrow Y$ with $X \cap Y = \emptyset$ is characterized by:

$$\text{Support: } \text{supp}(X \Rightarrow Y) = \text{supp}(X \cup Y)$$

$$\text{Confidence: } \text{conf}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)}$$

$$\text{Lift: } \text{lift}(X \Rightarrow Y) = \frac{\text{conf}(X \Rightarrow Y)}{\text{supp}(Y)}$$

$$\text{Interest (leverage): } \text{interest}(X \Rightarrow Y) = \text{supp}(X \cup Y) - \text{supp}(X) \text{supp}(Y)$$

In practice, the main bottleneck is counting candidate pairs, since their number can be very large even when only few larger itemsets are frequent. The objective is to find all subsets $X \subseteq I$ that appear in at least a given number of movies (`minsup`), and then derive association rules $X \Rightarrow Y$ with high support, confidence, and lift. For example, in the actor-only setting a rule could be

$$\{\text{actor:daniel radcliffe}\} \Rightarrow \{\text{actor:emma watson}\},$$

indicating a strong co-casting pattern. In the enriched setting, a rule such as

$$\{\text{actor:tom hanks}\} \Rightarrow \{\text{gross_tier:high}\}$$

captures an actor–success association.

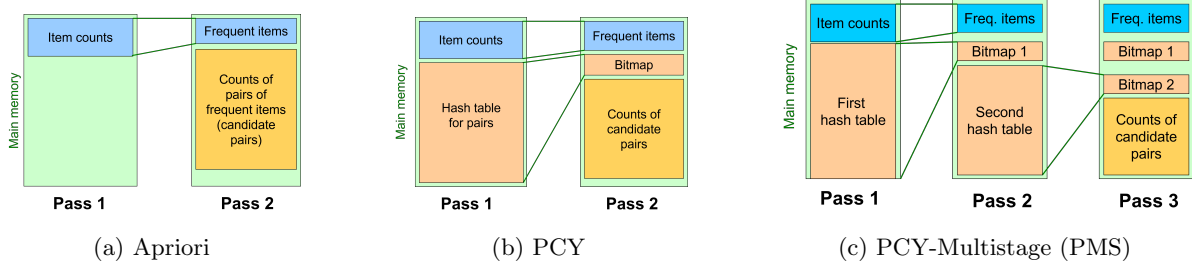
Because the number of possible itemsets is $2^{|I|}$, I rely on three classical ARM algorithms to prune this search space efficiently on movie baskets.

Across all experiments the workflow is the same (see Figure 4):

- Build movie baskets from the cleaned IMDB table
- Mine frequent singletons and pairs (2-itemsets) with Apriori, PCY, or PCY-Multistage
- Extend to larger itemsets ($k \geq 3$) using Apriori’s join-and-prune step
- Generate association rules from the union of all frequent itemsets.

The pair-mining phase is the main computational bottleneck, so Section 4 compares algorithms primarily in terms of the number of candidate pairs they actually count (C_2) and the corresponding

Figure 4: Main-memory flow of Apriori, PCY, and PCY-Multistage for pair mining on movie baskets.



runtimes, on both actor-only and enriched movie baskets.

3.2.1 Apriori on Movie Baskets

Apriori is the classical level-wise frequent-itemset algorithm based on the downward-closure property: if an itemset X is frequent, then every subset of X must also be frequent. In the movie context this means, for example, that if the triple $\{\text{actor:daniel radcliffe}, \text{actor:emma watson}, \text{actor:rupert grint}\}$ is frequent, then each pair from this triple must also be frequent.

I implement the standard two-pass version for pairs over the actor-only and enriched baskets:

- **Pass 1 (frequent items):** scan all movie baskets once and count each item (actor or actor/feature token), then keep items with support $\geq \text{minsup}$ as L_1 .
- **Pass 2 (frequent pairs):** for each basket, restrict to L_1 , generate unordered pairs $\{i, j\}$, and count their supports; keep pairs with support $\geq \text{minsup}$ as L_2 .

For itemsets of size $k \geq 3$, I use the textbook Apriori join-and-prune strategy (MMDS Algorithm 6.1): candidates C_k are built by joining frequent $(k-1)$ -itemsets in L_{k-1} , removing candidates whose $(k-1)$ -subsets are not all in L_{k-1} , then counting supports over all movie baskets to obtain L_k . The detailed Apriori algorithm is shown below.

Apriori pseudocode (run_apriori_full)

Input: transaction database D , minimum support ϵ , maximum size K .

Output: frequent itemsets $L[k]$ and support counts $\text{Cnt}[k]$.

```

01 Count all items in  $D$  to get singleton supports.
02 Build  $L_1 = \{\{i\} \mid \text{support}(i) \geq \epsilon\}$ ; set  $L[1] \leftarrow L_1$ ,  $\text{Cnt}[1] \leftarrow$  singleton counts.
03 If  $K > 1$ , generate all 2-item combinations from items in  $L_1$  as  $C_2$ .
04 Count supports of  $C_2$  over all transactions and keep  $L_2 = \{c \in C_2 \mid \text{support}(c) \geq \epsilon\}$ .
05 Store  $L[2] \leftarrow L_2$ ,  $\text{Cnt}[2] \leftarrow$  pair counts.
Method: 06 For  $k = 3$  to  $K$ : if  $L[k-1]$  is empty, stop.
07 Generate  $C_k$  by Apriori join-and-prune from  $L[k-1]$ .
08 If  $C_k$  is empty, stop.
09 Count supports of all candidates in  $C_k$ , then keep  $L_k = \{c \in C_k \mid \text{support}(c) \geq \epsilon\}$ .
10 Store  $L[k] \leftarrow L_k$ ,  $\text{Cnt}[k] \leftarrow$  counts of  $C_k$ .
11 Return all  $L[k]$  and  $\text{Cnt}[k]$ .

```

For each configuration (basket design, minsup , max k) I record:

- $|L_1|$: number of frequent single items,
- C_2^{Apr} : number of distinct pairs whose support is counted in Pass 2,

- $|L_2^{\text{Apr}}|$: number of frequent pairs found,
- **time_Apr**: runtime of pair mining (Passes 1–2).

Higher-order counts $|L_k|$ for $k = 3, 4$ are also reported, but the main comparison with PCY and PCY-Multistage focuses on C_2 and runtime.

3.2.2 PCY: Hashing Pairs in Pass 1

PCY (Park–Chen–Yu) is a memory-efficient refinement of Apriori that uses a hash-bucket bitmap to filter impossible candidate pairs before they are counted. Given a hash function h_1 from unordered pairs to $\{0, \dots, B-1\}$, the implementation is:

- Pass 1 (frequent items + bucket counts): as in Apriori, scan all movies to obtain L_1 ; at the same time, hash every observed pair $\{i, j\}$ into bucket $h_1(i, j)$ and increment bucket counts. After the scan, create Bitmap 1 where bucket b is marked frequent if its count is $\geq \text{minsup}$.
- Pass 2 (filtered pair counting): for each movie, restrict to L_1 , generate pairs, and count a pair $\{i, j\}$ only if its bucket $h_1(i, j)$ is marked frequent in Bitmap 1.

The key intuition for this movie data is that many actor combinations (especially in enriched baskets) are extremely rare. If a bucket total is below **minsup**, then every specific pair hashing into that bucket must be infrequent, so all those pairs can be skipped safely.

PCY pseudocode (pcy_full)

Input: transaction database D , minimum support ϵ , maximum size K , bucket count B .

Output: frequent itemsets $L[k]$, support counts $\text{Cnt}[k]$, and stage-1 bitmap.

- 01 Initialize singleton counter and stage-1 bucket counters of size B .
- 02 Pass 1 over D : update singleton counts and hash each observed pair (i, j) to bucket $h_1(i, j)$.
- 03 Build $L_1 = \{i \mid \text{support}(i) \geq \epsilon\}$ and $\text{bitmap}_1[b] = 1$ if bucket count $b \geq \epsilon$, else 0.
- 04 Pass 2 over D : for each transaction, keep only items in L_1 , generate pairs, and count a pair only if $\text{bitmap}_1[h_1(i, j)] = 1$.
- Method:** 05 Build $L_2 = \{p \mid \text{support}(p) \geq \epsilon\}$.
- 06 Convert L_1 and L_2 to set-form itemsets; store in $L[1]$, $L[2]$, with counts in $\text{Cnt}[1]$, $\text{Cnt}[2]$.
- 07 For $k = 3$ to K : if $L[k-1]$ is empty, stop.
- 08 Generate C_k from $L[k-1]$ using Apriori join-and-prune; if empty, stop.
- 09 Count supports of C_k over D , keep frequent L_k , and store in $L[k]$, $\text{Cnt}[k]$.
- 10 Return $L[k]$, $\text{Cnt}[k]$, and bitmap_1 .

For each experiment I collect:

- C_2^{PCY} : number of distinct pairs that pass Bitmap 1 and are counted in Pass 2,
- $|L_2^{\text{PCY}}|$: number of frequent pairs returned (empirically matching Apriori),
- **time_PCY**: runtime of pair mining.

3.2.3 PCY-Multistage: Two Hash Filters

PCY-Multistage (PMS) extends PCY by applying two independent hash filters before final pair counting. This adds a second screening step for pairs that survive the first bitmap due to collisions.

Let h_1 and h_2 be two hash functions from unordered pairs to $\{0, \dots, B-1\}$

- Pass 1 (Bitmap 1): compute L_1 , hash observed pairs with h_1 , and build Bitmap 1.

- Pass 2 (Bitmap 2): consider only pairs with items in L_1 and frequent bucket in Bitmap 1; re-hash them with h_2 , then build Bitmap 2.
- Pass 3 (final pair counting): count a pair $\{i, j\}$ only if both items are in L_1 , $h_1(i, j)$ maps to a frequent bucket in Bitmap 1, and $h_2(i, j)$ maps to a frequent bucket in Bitmap 2.

This reduces false-positive pair candidates from hash collisions. A pair must survive two independent bitmap checks before exact counting, which typically shrinks the counted candidate set more than single-stage PCY.

PCY-Multistage pseudocode (pcy_multistage_full)

Input: transaction database D , minimum support ϵ , maximum size K , stage-1 buckets B_1 , stage-2 buckets B_2 .

Output: frequent itemsets $L[k]$, support counts $\text{Cnt}[k]$, Bitmap 1, Bitmap 2.

- 01 Define two stage-specific hash functions $h_1(i, j)$ and $h_2(i, j)$.
- 02 Pass 1: count singletons and stage-1 bucket totals; build L_1 and Bitmap 1 (bucket frequent if count $\geq \epsilon$).
- 03 Pass 2: for each transaction, keep unique items in L_1 ; for each pair, keep it only if Bitmap 1 accepts it, then hash with h_2 and update stage-2 buckets.
- 04 Build Bitmap 2 from stage-2 bucket counts using the same threshold ϵ .
- Method:** 05 Pass 3: scan transactions again; count a pair only if it passes all three checks: both items in L_1 , Bitmap 1 acceptance, and Bitmap 2 acceptance.
- 06 Build $L_2 = \{p \mid \text{support}(p) \geq \epsilon\}$, then store $L[1]$, $L[2]$, $\text{Cnt}[1]$, $\text{Cnt}[2]$.
- 07 For $k = 3$ to K : if $L[k-1]$ is empty, stop.
- 08 Generate C_k with Apriori join-and-prune; if empty, stop.
- 09 Count supports of C_k , keep frequent L_k , and store in $L[k]$, $\text{Cnt}[k]$.
- 10 Return $L[k]$, $\text{Cnt}[k]$, Bitmap 1, and Bitmap 2.

For each configuration I track:

- C_2^{PMS} : number of distinct pairs that survive both bitmaps and are counted in Pass 3,
- $|L_2^{\text{PMS}}|$: number of frequent pairs found (empirically matching Apriori and PCY),
- `time_PMS`: runtime of the three-pass pair-mining stage.

Section 4 compares C_2 and runtime side by side for Apriori, PCY, and PMS on both actor-only baskets (four actors per movie) and enriched baskets (actors plus tier items), to quantify the practical gain from one-stage and two-stage hashing over the Apriori baseline.

3.3 Features Engineering

3.3.1 Actors Variable

The main modeling decision for frequent-itemset mining is to treat each movie as a basket and its main actors as items. I construct actor-only baskets as follows:

- Normalize actor names in `star1 : star4`: strip whitespace, lowercase, and prefix with `actor:` (e.g., “Tom Hanks” $\rightarrow$ `actor:tom hanks`).
- For each movie, collect these four names, drop NaNs (none in this dataset), and remove duplicates within the movie, then store them as an unordered set.
- Discard movies that would end up with an empty basket (none here).

This yields an actor-only basket dataset with:

- 1000 baskets (movies),

- exactly 4 items per basket in almost all cases,
- several thousand distinct actor items in the global universe.

(See Table 5 and Table 6.)

Table 5: Top 10 Actors by Appearance Frequency

Rank	Actor	Count
1	Robert De Niro	17
2	Tom Hanks	14
3	Al Pacino	13
4	Brad Pitt	12
5	Clint Eastwood	12
6	Christian Bale	11
7	Leonardo DiCaprio	11
8	Matt Damon	11
9	James Stewart	10
10	Humphrey Bogart	9

Table 6: Basket-Level Summary for Actor–Movie Data

Metric	Value
n_baskets	1000
n_distinct_actors	2709
basket_size_min	3
basket_size_mean	4.00
basket_size_max	4
max_actor_frequency	17
actors_with_ge_8_movies	17

3.3.2 Enriched variables

To study how the algorithms behave when the itemspace is larger and baskets contain heterogeneous features, I also construct an enriched basket representation. For each movie I retain the four actor items and add one discretized tier item for each of several numeric variables:

- `gross_tier` (e.g., low/mid/high by quantiles),
- `rating_tier` (buckets of IMDB rating),
- `meta_score_tier`,
- `votes_tier`,
- `runtime_tier`.

Each tier is encoded as an item like `gross_tier:high` or `votes_tier:low`, so a typical enriched basket contains 4 actor items plus 4–5 tier items. The enriched basket space therefore has:

- the same 1000 movies,
- a significantly larger item universe (actors + feature tiers),
- larger average basket size and a much denser candidate pair space.

Table 7: Enriched Basket and Tier Item Summary

Metric	Value
n_baskets	1000
items_universe	2724
basket_size_min	7
basket_size_mean	8.67
basket_size_max	9
top_tier_item_1	rating_tier:low (431)
top_tier_item_2	runtime_tier:low (343)
top_tier_item_3	runtime_tier:mid (339)
top_tier_item_4	votes_tier:low (334)
top_tier_item_5	votes_tier:high (333)
top_tier_item_6	votes_tier:mid (333)
top_tier_item_7	rating_tier:high (322)
top_tier_item_8	runtime_tier:high (318)
top_tier_item_9	meta_score_tier:low (284)
top_tier_item_10	meta_score_tier:mid (282)

This enriched representation is used solely to stress-test candidate generation, as discussed in the next section, showing that the relative advantage of PCY and especially PCY-Multistage in terms of C_2 reduction and runtime becomes much more visible when the item universe and basket size increase, even though the underlying IMDB data remain the same.

4 Experimental Setup and Results

This section describes how I configure and run Apriori, PCY, and PCY-Multistage on IMDB movie baskets, and how candidate counts and runtimes are used to compare configurations for each basket design. All experiments are executed in a single Python notebook (`experiment-code-v4.0`) with shared preprocessing and fixed reproducibility settings.

4.1 Reproducibility and Common Settings

All experiments share the same environment settings:

- Dataset and preprocessing: cleaned IMDB Top-1000 table from Sections 1–4, with normalized actor names and engineered tier features.
- Random seed: `SEED = 42` for Python `random` and NumPy.
- Bucket configuration: `DEFAULT_NUM_BUCKETS = 2048` for PCY and PCY-Multistage (both hash stages in PMS use 2048 buckets).
- Timing protocol: wall-clock time measured with `time.perf_counter` around pair-mining passes only, so one-off data loading/cleaning does not affect algorithm comparison.

4.2 Basket Designs and Minsup Grids

Actor-only baskets

For the actor-only design, each movie is represented as a basket of up to four actors (`star1–star4`) after cleaning and normalization as reported in Table 6. I use a relative support grid $\{0.002, 0.003, 0.005, 0.007\}$, which maps to absolute minsup values $\{2, 3, 5, 7\}$ since $m = 1000$, and cap Apriori-style expansion at $k_{\max} = 4$ to match the maximum basket width.

Enriched baskets

For the enriched design, each movie basket contains the actor items plus five compact tier features (`gross_tier`, `votes_tier`, `rating_tier`, `runtime_tier`, `meta_score_tier`), which increases basket width and the density of the pair space (see Table 7). Here I use a broader support grid aligned with the notebook outputs: absolute minsup values $\{2, 5, 10, 30, 50\}$, corresponding to ratios $\{0.002, 0.005, 0.010, 0.030, 0.050\}$, and allow Apriori-style expansion up to $k_{\max} = 9$, which is the maximum observed enriched basket size.

Evaluation metrics and selection criterion

4.2.1 Metrics

For each dataset, support threshold, and algorithm, I record:

- C_2 : number of candidate pairs actually counted during the pair-mining phase.
- $|L_2|$: number of frequent pairs returned at that minsup.
- `time_pair`: pair-mining runtime (Apriori passes 1–2, PCY passes 1–2, PMS passes 1–3).

As a consistency check, I also track $|L_k|$ and C_k for $k \geq 3$ from the Apriori expansion, to verify that higher-order behavior matches the expected sparsity pattern.

4.2.2 Selection rule

To compare configurations, I apply the following decision rule:

1. Retain only configurations that reproduce the same $|L_2|$ as Apriori at the same minsup (correctness constraint).
2. Among those, prefer the configuration with smaller C_2 (stronger candidate pruning).
3. If C_2 ties, choose the configuration with lower `time_pair`.

The definitions and formulas for the underlying rule-quality measures are summarized in Table 8.

4.3 Experiment Results

4.3.1 Algorithm behavior on actor-only baskets

On actor-only baskets, Apriori and PCY always produce identical frequent itemsets L_1 and L_2 at a given support threshold (Table 9), which confirms that hash-based pruning in PCY does not change

Table 8: Summary of Association Rule Evaluation Metrics

Metric	Definition	Formula / computation
Support	Fraction of baskets containing X	$\text{supp}(X) = \frac{\text{supp_count}(X)}{m}$
Confidence	Fraction of X -baskets that also contain Y	$\text{conf}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)}$
Interest (leverage)	Excess co-occurrence above what independence predicts	$\text{interest}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y) - \text{supp}(X)\text{supp}(Y)}{\text{supp}(X)\text{supp}(Y)}$
Lift	How many times more often X and Y co-occur than under independence	$\text{lift}(X \Rightarrow Y) = \frac{\text{conf}(X \Rightarrow Y)}{\text{supp}(Y)} = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)\text{supp}(Y)}$

correctness. The difference lies in how many candidates each method has to examine: while both algorithms consider all 2709 items as candidates in C_1 , PCY consistently reduces the number of candidate pairs C_2 relative to Apriori (Table 10), with reductions growing from about 5% at $\text{minsup} = 2$ to more than 90% at $\text{minsup} = 7$.

PMS (Multistage PCY) pushes this idea further by using a second hash stage, and the effect is visible in Table 11: at $\text{minsup} = 5$, it reduces C_2 from 139 (Apriori) to only 3 candidate pairs, i.e., it is almost counting only truly frequent pairs. Table 12 and Figure 5 summarize these candidate-level savings: PCY removes a modest but consistent fraction of the Apriori candidates, whereas PMS removes a large majority of them at low supports, especially when $\text{minsup_ratio} \leq 0.005$.

These reductions translate into large runtime effects. As shown in Table 13 and Figure 6, both PCY and PMS achieve orders-of-magnitude speedups over Apriori at low support for example, at $\text{minsup_ratio} = 0.002$, PCY and PMS are more than 10^2 – 10^3 times faster than Apriori—while speedups shrink to single-digit factors when support is higher and the candidate space is small. Conceptually, this behavior matches the design of the algorithms: when many pairs are potential candidates, hash-based pruning (PCY, PMS) pays off; when few remain, the overhead of hashing and extra passes dominates.

Table 9: Apriori vs. PCY on Actor-Only Baskets: Frequent Itemsets L_k

minsup_ratio	minsup	L1_Apr	L2_Apr	L1_PCY	L2_PCY
0.002	2	641	121	641	121
0.003	3	271	25	271	25
0.005	5	79	3	79	3
0.007	7	30	0	30	0

Table 10: Apriori vs. PCY on Actor-Only Baskets: Candidate Itemsets C_k

minsup_ratio	minsup	C1_Apr	C2_Apr	C1_PCY	C2_PCY	C2 drop	C2 drop (%)
0.002	2	2709	1516	2709	1439	77	5.08
0.003	3	2709	622	2709	501	121	19.45
0.005	5	2709	139	2709	51	88	63.31
0.007	7	2709	32	2709	3	29	90.62

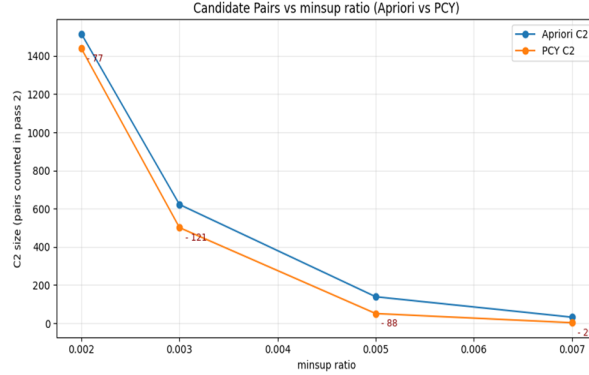
Table 11: PMS Candidate Reduction at the Pair Level (Actor-Only Baskets)

minsup_ratio	minsup	C1_Apr	C2_Apr	C1_PMS	C2_PMS	C2 drop	C2 drop (%)
0.002	2	2709	1516	2709	795	721	47.56
0.003	3	2709	622	2709	69	553	88.91
0.005	5	2709	139	2709	3	136	97.84
0.007	7	2709	32	2709	0	32	100.00

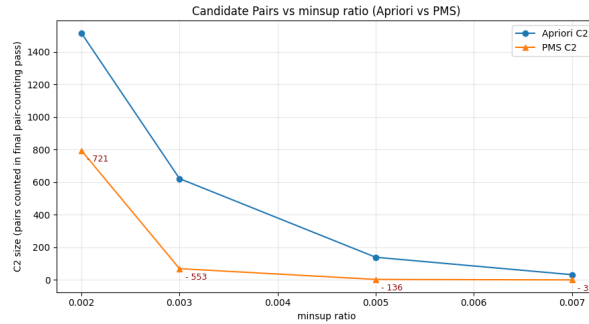
Table 12: PCY Candidate Reduction Relative to Apriori (Actor-Only Baskets)

minsup_ratio	minsup	C2_Apr	C2_PMS	C2 drop	C2 drop (%)	C3_Apr	C3_PCY
0.002	2	1516	1439	77	5.08	26	26
0.003	3	622	501	121	19.45	4	4
0.005	5	139	51	88	63.31	1	1
0.007	7	32	3	29	90.62	0	0

Figure 5: Comparison of Candidate generation reduction (Actor-only Baskets). (a) PCY vs Apriori (b) PMS vs Apriori



(a) Candidate generation reduction of PCY relative to Apriori.

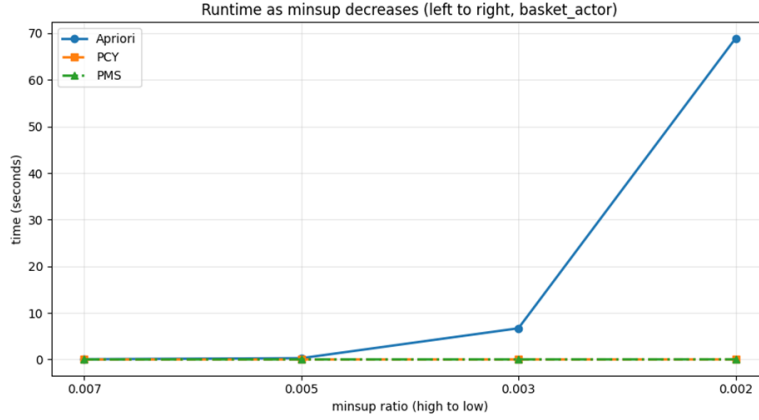


(b) Candidate generation reduction of PMS relative to Apriori

Table 13: Runtime and Speedup of PCY and PMS Relative to Apriori (Actor-only Baskets)

minsup_ratio	minsup	time_Apriori (s)	time_PCY (s)	time_PMS (s)	speedup_PCY / Apriori	speedup_PMS / Apriori
0.002	2	90.69	0.0398	0.1950	2277.08	465.10
0.003	3	18.21	0.0403	0.0340	452.25	535.44
0.005	5	0.79	0.0237	0.0289	33.34	27.31
0.007	7	0.13	0.0206	0.0336	6.43	3.94

Figure 6: Runtime performance at different support threshold (Apriori vs PCY vs PMS) (Actor-only Baskets)



4.3.2 Algorithm behavior on enriched baskets

On enriched baskets (actors plus a small number of movie attributes), the candidate space at low support grows much larger, making pruning more critical. Table 14 shows that at minsup = 2 all methods face 7471 candidate pairs, but PMS begins to deliver substantial savings once support increases: at minsup = 5 it cuts C_2 from 1230 (Apriori) to 547, and at minsup = 10 it still removes roughly 42% of Apriori’s candidates. The combined candidate trends for Apriori, PCY, and PMS on enriched baskets are visualized in Figure 7a, which highlights that multistage hashing is most beneficial in regimes where many pairs are close to the support threshold.

Runtime results in Table 15 and Figure 7b mirror these patterns. At minsup_ratio = 0.002, both PCY and PMS are roughly 4 times faster than Apriori, reflecting the large candidate reductions, while at medium to high support the three algorithms converge to similar runtimes and PCY/PMS can even be slightly slower because the cost of hashing outweighs the smaller number of counted pairs. This behavior is consistent with the theoretical expectation that hashing is most advantageous when memory pressure from C_2 is high.

4.3.3 Final configuration

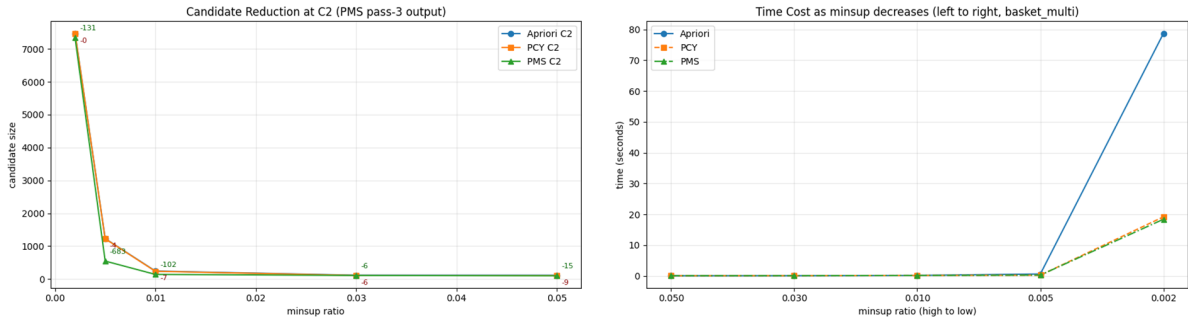
Table 14: Candidate Reduction on Enriched Baskets

minsup_ratio	minsup	C2 (Apriori)	C2 (PCY)	C2 (PMS)	PMS Reduction (%)
0.002	2	7471	7471	7340	1.75%
0.005	5	1230	1226	547	55.53%
0.010	10	243	236	141	41.98%
0.030	30	115	109	109	5.22%
0.050	50	115	106	100	13.04%

Table 15: Runtime and Speedup of PCY and PMS Relative to Apriori (Enriched Baskets)

minsup_ratio	minsup	time_Apriori (s)	time_PCY (s)	time_PMS (s)	speedup_PCY / Apriori	speedup_PMS / Apriori
0.002	2	78.71	19.18	18.39	4.10	4.28
0.005	5	0.63	0.24	0.26	2.62	2.42
0.010	10	0.15	0.15	0.15	0.98	1.00
0.030	30	0.07	0.08	0.09	0.86	0.78
0.050	50	0.05	0.06	0.07	0.88	0.76

Figure 7: Enriched-basket comparison: candidate reduction and runtime behavior.



(a) Candidate generation reduction of PMS/PCY relative to Apriori (Enriched Baskets).

(b) Runtime comparison between Apriori and PCY (Enriched Baskets).

Taken together, these experiments support the following configuration choices. On actor-only baskets, all three algorithms are correct but differ in efficiency: PCY reduces the candidate space significantly and delivers the best runtime across support thresholds, while PMS yields the most aggressive C_2 pruning at the price of an extra pass and only modest additional speedup on this dataset size. On enriched baskets, multistage hashing (PMS) becomes more attractive at low support because the candidate space is much larger; however, once support is moderate or high, the runtime advantage over PCY and even Apriori largely disappears.

Given these observations and the underlying algorithmic trade-offs (monotonicity-based pruning in Apriori versus hash-based pruning in PCY/PMS), I adopt PCY on actor-only baskets as the default configuration for the final rule-mining stage. Concretely, I fix `minsup_ratio` = 0.005 (with absolute `minsup` derived from the number of movies m) and `minconf` = 0.50, which provides enough support for interpretable rules while keeping frequent-pair mining computationally tractable (as in Table 16b).

Table 16: Final Configurations for Association Rule Mining

(a) Actor-only baskets		(b) Enriched baskets	
Setting	Choice	Setting	Choice
Baskets	Actor-only (movies $\rightarrow$ sets of actors)	Baskets	Enriched (actors + selected movie attributes)
Algorithm	PCY	Algorithms	Apriori, PCY, PMS
<code>minsup_ratio</code>	0.005	<code>minsup_ratio</code>	{0.002, 0.005, 0.01, 0.03, 0.05}
<code>minsup</code> (absolute)	$\lfloor 0.005 \cdot m \rfloor$	<code>minsup</code> (absolute)	$\lfloor \text{minsup_ratio} \cdot m \rfloor$
<code>minconf</code>	0.50	<code>minconf</code>	0.50 (for rule-mining runs)

5 Association Rules Mining

5.1 Rule Mining Configuration

The final rule-mining configuration used for reporting is fixed to Table 17:

Table 17: Rule-Mining Configuration

Baskets	Algorithm	<code>minsup_ratio</code>	<code>minsup</code>	<code>minconf</code>
Actor-only	PCY	0.005	5	0.5
Enriched	PMS	0.05	50	0.5

5.2 Rule Metrics

Rules are then generated from frequent itemsets mined under this setup, following the classical association-rule framework (Yan et al., 2009), the notebook computes:

- **Support** (joint frequency):

$$\text{supp}(X \Rightarrow Y) = \text{supp}(X \cup Y) = \frac{|\{i : X \cup Y \subseteq T_i\}|}{m}.$$

- **Confidence** (directional reliability, $P(Y | X)$):

$$\text{conf}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)}.$$

- **Lift** (association strength vs. independence):

$$\text{lift}(X \Rightarrow Y) = \frac{\text{conf}(X \Rightarrow Y)}{\text{supp}(Y)} = \frac{\text{supp}(X \cup Y)}{\text{supp}(X) \text{supp}(Y)}.$$

- **Interest (leverage)** (absolute deviation from independence) (“interestMeasure”, n.d.):

$$\text{interest}(X \Rightarrow Y) = \text{supp}(X \cup Y) - \text{supp}(X) \text{supp}(Y).$$

- **Kulczynski (KULC)** (average of directional confidences):

$$\text{Kulc}(X, Y) = \frac{1}{2}(P(Y | X) + P(X | Y)),$$

where values near 1 or 0 indicate strong positive or negative association and values near 0.5 indicate weak or uninteresting rules.

- **Fisher’s exact test** (“Statistical Notes for Clinical Researchers”, n.d.) To reduce the chance of spurious rules, I additionally apply a one-sided Fisher’s exact test (alternative = “greater”) on the 2×2 contingency table

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} |X \cap Y| & |X \cap \neg Y| \\ |\neg X \cap Y| & |\neg X \cap \neg Y| \end{bmatrix},$$

and keep only rules with $p < 0.05$, which indicates that X and Y co-occur significantly more often than expected under independence.

All above-mentioned metrics follow standard rule-interestingness practice in data mining and are summarized in Table 8 and from previous work of “Tan, P.-N., Steinbach, M. and Kumar, V. (2005) Introduction to Data Mining. Pearson Addison Wesley, Boston. - References - Scientific Research Publishing” in Introduction to Data Mining, 2008.

5.3 Filtering and ranking policy

Based on previous work in “Mining Top K Non-redundant Association Rules” of Fournier-Viger and Tseng, 2012, I adopt the “interest for filtering, lift for ranking” strategy that first enforces hard constraints and then orders the remaining rules:

Table 18: Filtering and ranking policy for association rules

No.	Step	Description
1	Filter by support	Discard rules with $\text{supp}(X \cup Y) < s_{\min}$ to ensure each rule is based on a sufficient number of baskets.
2	Filter by confidence	Discard rules with $\text{conf}(X \Rightarrow Y) < c_{\min}$ so that the consequent occurs often enough when the antecedent is present.
3	Filter by interest	Keep only rules with $ \text{interest}(X \Rightarrow Y) \geq \iota_{\min}$, removing rules that are explained purely by the marginal popularity of X or Y .
4	Filter by Fisher significance	Apply Fisher’s exact test and discard rules with two-sided $p \geq \alpha$ (here $\alpha = 0.05$), retaining only statistically significant departures from independence.
5	Remove redundant rules	Prune rules for which there exists a more specific rule $X' \Rightarrow Y'$ with $X \subset X'$, $Y \subseteq Y'$, and identical support and confidence; such rules do not add new information.
6	Apply maximal antecedent filter	For a fixed consequent Y , keep only rules whose antecedent X is inclusion-maximal among non-redundant rules with comparable support/confidence, so that each reported rule uses the largest informative context for predicting Y .
7	Ranking (after filtering)	On the surviving set, rank rules primarily by lift, breaking ties by higher confidence and then by higher interest to surface rules that are strongly associated (high lift), reliable (high confidence), and substantially above chance co-occurrence (high interest).

On the surviving set, I rank rules primarily by lift, breaking ties by higher confidence and then by higher interest, which surfaces rules that are both strongly associated (high lift), reliable (high confidence), and substantially above chance co-occurrence (high interest).

In practice, I read rules by first checking support (sufficient number of examples), then confidence, then interest and KULC to rule out popularity effects, and finally the Fisher p -value, before applying redundancy and maximality filters and ranking the remaining rules for reporting. In the experiments, I retain only rules that satisfy $\text{supp_count}(X \cup Y) \geq s_{\min}$ and $\text{conf}(X \Rightarrow Y) \geq c_{\min}$ and that have a one-sided Fisher p -value below 0.05, so that the reported rules are frequent enough, sufficiently strong, and statistically significant under an independence null model.

5.4 Rules Analysis

5.4.1 Actor-only Baskets

Before analyzing individual rules, I fix a single PCY configuration on actor-only baskets and summarize it in Table 19. With $\text{minsup_ratio} = 0.005$ (i.e., $\text{minsup} = 5$ movies), 79 actors remain frequent in L_1 ; PCY then generates 51 candidate pairs C_2 , of which only 3 pairs are frequent in L_2 , all found in 0.026 seconds. Using hash buckets ensures that most non-promising pairs are pruned at the hashing stage while preserving all truly frequent actor–actor pairs for subsequent rule mining.

For actor-only baskets, the strongest rules are pure actor–actor co-occurrences and recover well-known franchise structure. The top five rules (Table 20) all involve the Harry Potter trio (Daniel Radcliffe, Rupert Grint, Emma Watson) with support between 0.005 and 0.006, confidence between 0.714 and 1.000, and extremely high lift (approximately 120–167), indicating that whenever one of these actors appears, the others are present far more often than chance would predict. The frequent triple $\{\text{Radcliffe},$

Table 19: Configuration summary of association-rule mining run (Actor-only Baskets)

Quantity	Value
minsup_ratio	0.005
minsup	5
L1_size	79
C2_size	51
L2_size	3
time_sec	0.02583
num_buckets	2048
freq_buckets	365

Watson, Grint appears in 5 of 1000 movies, exactly matching the main Harry Potter entries in the Top-1000 subset, and illustrates that the actor-only configuration captures franchise-level patterns with very high directional reliability.

Table 20: Top 5 actor-actor rules in actor-only baskets

lhs	rhs	support_count	support	confidence	lift	interest
daniel radcliffe	rupert grint	6	0.006	1.000000	166.666667	0.005964
rupert grint	daniel radcliffe	6	0.006	1.000000	166.666667	0.005964
daniel radcliffe	emma watson	5	0.005	0.833333	119.047619	0.004958
rupert grint	emma watson	5	0.005	0.833333	119.047619	0.004958
emma watson	daniel radcliffe	5	0.005	0.714286	119.047619	0.004958

Table 21: Frequent actor triples (L3) in actor-only baskets

triple	support_count	support
(daniel radcliffe, emma watson, rupert grint)	5	0.005

5.4.2 Enriched Baskets

For enriched baskets, I move from actor-only baskets to baskets that combine actors with discretized success features (gross, votes, rating, runtime, metascore tiers) and run PMS with two support settings (Config A and Config B in Table 22). Config A uses minsup_ratio = 0.01 (minsup = 10 movies) to expose more candidate patterns, while Config B uses a stricter minsup_ratio = 0.05 (minsup = 50 movies) to focus on highly supported rules; both share minconf = 0.5, $k_{\max} = 9$, 4,096 hash buckets, and a p -value threshold of 0.05.

Table 23 shows how the filtering pipeline compresses the rule set at these two supports. At minsup_ratio = 0.01, 475 rules pass the basic configuration; Fisher’s test removes only a small fraction (retaining 97.7%), but redundancy and maximality pruning are much stronger, leaving 160 final rules (about 34% of the original). At minsup_ratio = 0.05 the search starts from only 46 rules, all of which are significant by Fisher; the structural filters again remove many near-duplicates, leaving 31 final rules (about 67% retention), with most surviving rules being multi-item antecedents targeting a small number of consequents.

Within this setup, the clearest single-actor signal is **actor:tom hanks** $\Rightarrow$ **gross_tier:high** (Table 24). The rule has support 14/1000, confidence 1.0, lift 3.61, interest 0.0101, a two-sided Fisher

Table 22: Association-rule mining configurations

Parameter	Config A	Config B
n_baskets	1000	1000
minsup_ratio	0.01	0.05
minsup	10	50
minconf	0.5	0.5
max_k	9	9
num_buckets	4096	4096
p_threshold	0.05	0.05

Table 23: Rule counts at successive filtering stages

Stage	minsup_ratio = 0.01		minsup_ratio = 0.05	
	Rules	Retention (%)	Rules	Retention (%)
raw_after_configuration	475	100.00	46	100.00
after_fisher_test_significance	464	97.68	46	100.00
after_non_redundant	300	63.16	37	80.43
after_maximal (final rules)	160	33.68	31	67.39
final_l1_to_l1	7	1.47	1	2.17
final_n_to_l1	153	32.21	30	65.22

p -value around 1.2×10^{-8} , and Kulczynski ≈ 0.53 , indicating a strong, statistically significant association between Tom Hanks and high-gross movies beyond what the marginal frequencies would suggest. The supporting titles *Forrest Gump*, *Saving Private Ryan*, *The Green Mile*, *Toy Story*, *Toy Story 3* are all well-known high-revenue films, which makes this rule a sanity check that the enriched configuration can recover intuitive star-success relationships.

Table 24: Top One-Item 1 $\rightarrow$ 1 rule for `gross_tier:high` and supporting movies

Antecedent	Consequent	supp_cnt	supp	conf	lift	interest	p-value	Kulc
actor:tom hanks	gross_tier:high	14	0.014	1.0	3.610108	0.010122	1.23×10^{-8}	0.525271

Title	Year
Forrest Gump	1994
Saving Private Ryan	1998
The Green Mile	1999
Toy Story	1995
Toy Story 3	2010

Multi-item rules for `gross_tier:high` further refine this picture by combining actor and feature information (Table 25). For instance, $\{\text{meta_score_tier:low}, \text{runtime_tier:high}\} \Rightarrow \text{gross_tier:high}$ has support 0.016, confidence 1.0, lift 3.61, and a very small p -value, suggesting that long, modestly reviewed films in this sample tend disproportionately to be high-gross blockbusters (e.g., *The Hobbit: An Unexpected Journey*, *Rogue One*, *Captain America: The Winter Soldier*).

Similarly, rules like:

$\{\text{actor:matt damon, votes_tier:high}\} \Rightarrow \text{gross_tier:high},$
 $\{\text{actor:leonardo dicaprio, votes_tier:high}\} \Rightarrow \text{gross_tier:high}.$

show that when these actors appear in highly voted films, those films are almost always in the high-gross tier (confidence 1.0, lift 3.61, highly significant p -values), reflecting a combination of star power and large audience reach.

Table 25: Top $N \rightarrow 1$ rules for **gross_tier:high** and example high-grossing movies

(a) Top 5 $N \rightarrow 1$ rules for **gross_tier:high**

Antecedent	Consequent	supp_cnt	supp	conf	lift	interest	p-value	Kulc
{meta_score_tier:low, time_tier:high}	run- gross_tier:high	16	0.016	1.000000	3.610108	0.011568	8.72×10^{-10}	0.528881
{actor:matt votes_tier:high}	damon, gross_tier:high	10	0.010	1.000000	3.610108	0.007230	2.36×10^{-6}	0.518051
{actor:leonardo votes_tier:high}	dicaprio, gross_tier:high	10	0.010	1.000000	3.610108	0.007230	2.36×10^{-6}	0.518051
{actor:leonardo time_tier:high}	dicaprio, run- gross_tier:high	10	0.010	1.000000	3.610108	0.007230	2.36×10^{-6}	0.518051
{runtime_tier:high, ing_tier:mid, votes_tier:high}	rat- gross_tier:high	13	0.013	0.928571	3.352243	0.009122	4.83×10^{-7}	0.487751

(b) Example high-grossing movies matching the top rule

Title	Year
<i>Movies matching top $N \rightarrow 1$ rule: {meta_score_tier:low, rating_tier:low, runtime_tier:high, votes_tier:high} $\rightarrow$ {gross_tier:high}</i>	
The Hobbit: An Unexpected Journey	2012
Rogue One	2016
Harry Potter and the Deathly Hallows: Part 1	2010
Captain America: The Winter Soldier	2014
Star Trek Into Darkness	2013

Taken together, the rule-mining configuration and filtering pipeline yield a small, interpretable set of statistically supported associations that align with domain intuition.

- *Actor-only baskets*: mined with PCY at low support, mainly recover very high-lift franchise co-occurrences such as the Harry Potter trio, showing that hashing-based pair mining can compress candidate sets without losing meaningful collaborations.
- *Enriched baskets*: extend this picture by combining actors with discretized success tiers, where the “interest for filtering, lift for ranking” strategy and Fisher’s test remove popularity-driven and redundant rules and highlight patterns like Tom Hanks’ consistent presence in high-gross titles and multi-feature signatures of blockbuster performance.

Overall, the experiments illustrate how classical interestingness measures, significance testing, and maximal non-redundant pruning can turn a large raw rule pool into a focused set of movie-success rules that are both statistically sound and easy to interpret.

6 Discussion and Conclusion

6.1 Key Empirical Takeaways

From the algorithm experiments, the main empirical finding is that PCY-style hashing behaves as predicted in the literature: it preserves the frequent pairs found by Apriori while reducing the number of counted candidate pairs (C_2), especially on enriched, higher-dimensional baskets. On actor-only baskets, this pruning is already visible in runtime at low support (for example, minsup = 2: Apriori $\approx$ 65.15s vs PCY $\approx$ 0.013s and PMS $\approx$ 0.014s), while gaps narrow as minsup increases. On enriched baskets (actors plus tier features), PCY and PCY-Multistage again produce smaller candidate sets and lower pair-mining runtimes under the same 2048-bucket budget.

On the rule-mining side, the final configured run (actor-only, PCY, minsup ratio 0.005, minconf 0.50) yields a compact and interpretable rule set. The surviving rules are mainly high-lift actor-actor links that match known collaborations, while stricter significance/redundancy/maximal filtering removes most weak or duplicated patterns. This supports the view that even this top-1000 IMDB sample contains non-trivial association structure.

6.2 Limitations of Data and Methods

The first limitation is dataset size. The IMDB Top-1000 subset has only 1000 baskets with four main actors per movie, so both the pattern space and the range of meaningful minsup values are limited. Although low-minsup runs can still be expensive for Apriori in this implementation, the dataset does not fully stress the true “massive data” regime discussed in MMDS (e.g., distributed I/O and cluster-scale memory pressure).

A second limitation is the inherent combinatorial hardness of frequent-itemset mining. The number of frequent itemsets can grow exponentially with the number of items, so no single method can avoid hard cases in general Arora and Barak, 2009. In practice, scalability depends on problem-specific choices: sensible minsup/minconf thresholds, strong pruning (downward closure, hashing, leverage-based filtering), and, for very large datasets, acceptance of heuristic or approximate methods when exact enumeration becomes too costly.

6.3 Possible Extensions and Future Work

A direct extension is larger data coverage (for example, a larger IMDB/TMDB corpus) to better evaluate memory usage, candidate growth, and runtime across broader minsup and bucket settings. With millions of baskets and a richer actor/feature vocabulary, the comparison between Apriori, PCY, and PCY-Multistage would be more representative of truly large-scale mining.

On the algorithmic side, FP-Growth is a strong follow-up baseline. Instead of explicit candidate generation, FP-Growth compresses transactions into an FP-tree and mines patterns through conditional pattern bases and depth-first recursion. Comparing FP-Growth against Apriori/PCY/PMS on the same movie baskets would clarify when tree-based compression outperforms hash-based pruning.

Another extension is graph-based analysis of mined associations: treat actors and feature tiers as nodes, and high-lift/high-interest relations as edges. Community detection, centrality, and motif analysis could reveal collaboration clusters and success-related structures that are less visible in rule tables alone.

6.4 Reproducibility

All experiments in this project are implemented in a single notebook and driven by a small set of global configuration knobs printed at the top (e.g., `SEED = 42`, `DEFAULT_NUM_BUCKETS = 2048`, and minsup grids for each dataset). The full pipeline from data loading and cleaning, through basket construction and algorithm comparison, to rule mining and post-processing runs end-to-end on the same cleaned IMDB dataset table, with intermediate tables and plots generated directly from code rather than edited by hand. This design, combined with fixed random seeds and explicit parameter logs, makes it straightforward to rerun all experiments on another machine, or to re-execute the notebook with different minsup/bucket settings or alternative datasets in future work.

Acknowledgement

I/We declare that this material, which I/We now submit for assessment, is entirely my/our own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my/our work, and including any code produced using generative AI systems. I/We understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me/us or any other person for assessment on this or any other course of study.

References

- Agrawal, R., Imielinski, T., & Swami, A. (1993, June 1). Mining Association Rules Between Sets of Items in Large Databases, SIGMOD Conference [Journal Abbreviation: Sigmod Record]. In *Sigmod Record* (p. 207, Vol. 22). <https://doi.org/10.1145/170036.170072>
- Agrawal, R., & Srikant, R. (2000). Fast Algorithms for Mining Association Rules. *Proc. 20th Int. Conf. Very Large Data Bases VLDB*, 1215.
- Arora, S., & Barak, B. (2009, April 20). *Computational Complexity: A Modern Approach* (1st ed.). Cambridge University Press. <https://doi.org/10.1017/CBO9780511804090>
- Data Mining Task - an overview — ScienceDirect Topics*. (n.d.). Retrieved March 16, 2026, from <http://www.sciencedirect.com/topics/computer-science/data-mining-task>
- Fournier-Viger, P., & Tseng, V. S. (2012). Mining Top-K Non-redundant Association Rules [Series Title: Lecture Notes in Computer Science]. In L. Chen, A. Felfernig, J. Liu, & Z. W. Raś (Eds.), *Foundations of Intelligent Systems* (pp. 31–40, Vol. 7661). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-34624-8_4
- Haripreethi, T., & Ananthanath, G. V. S. (2025). Enhancement Of Apriori Algorithm Using Hash Based Technique And Transaction Reduction. *12*(03).
- interestMeasure: Calculate Additional Interest Measures in arules: Mining Association Rules and Frequent Itemsets*. (n.d.). Retrieved March 16, 2026, from <https://rdr.io/cran/arules/man/interestMeasure.html>
- Mining of Massive Datasets*. (n.d.). ResearchGate. Retrieved March 16, 2026, from https://www.researchgate.net/publication/340676532_Mining_of_Massive_Datasets
- A primer to frequent itemset mining for bioinformatics*. (n.d.). ResearchGate. Retrieved March 16, 2026, from https://www.researchgate.net/publication/258103005_A_primer_to_frequent_itemset_mining_for_bioinformatics
- A primer to frequent itemset mining for bioinformatics. ResearchGate*. (n.d.). Retrieved March 16, 2026, from https://www.researchgate.net/publication/258103005_A_primer_to_frequent_itemset_mining_for_bioinformatics
- Srikant, R., & Agrawal, R. (1996). Mining sequential patterns: Generalizations and performance improvements [Series Title: Lecture Notes in Computer Science]. In P. Apers, M. Bouzeghoub, & G. Gardarin (Eds.), *Advances in Database Technology — EDBT '96* (pp. 1–17, Vol. 1057). Springer Berlin Heidelberg. <https://doi.org/10.1007/BFb0014140>
- Statistical notes for clinical researchers: Chi-squared test and Fisher's exact test*. (n.d.). ResearchGate. Retrieved March 16, 2026, from https://www.researchgate.net/publication/316944581_Statistical_notes_for_clinical_researchers_Chi-squared_test_and_Fisher's_exact_test
- Tan, P.-N., Steinbach, M. and Kumar, V. (2005) *Introduction to Data Mining*. Pearson Addison Wesley, Boston. - *References - Scientific Research Publishing*. (n.d.). Retrieved March 16, 2026, from <https://www.scirp.org/reference/referencespapers?referenceid=2569157>
- Yan, X., Zhang, C., & Zhang, S. (2009). CONFIDENCE METRICS FOR ASSOCIATION RULE MINING. *Applied Artificial Intelligence*, *23*(8), 713–737. <https://doi.org/10.1080/08839510903208062>